

專利資料庫分析工具架構摘要

1. 專案目標

本專案目標是建立一套「專利資料庫建置與專利侵權比對工具」，可將 WIPS Excel、GPSS API 或其他專利資料來源匯入後，經過原始保存、清理、去重合併、正式入庫與追溯記錄，優先支援主權項與獨立項的自動侵權比對，並由 HTML 前端操作、AI chat 協助分析，最後輸出可追溯的報告檔案。報表分析仍保留在架構中，但重要性低於侵權比對、匯入/匯出紀錄與報告生成；公司名稱標準化只作為報表統計與使用者端顯示的對照層，不改寫資料庫原始欄位值。

目前定位如下：

PostgreSQL 輕量正式資料庫

+ Docker 掛載式 Python 執行環境

+ Import / Export Audit Service

+ Claim Comparison / Infringement Analysis Service

+ Company Alias / Owner Display Mapping Service

+ Claude AI Service / AI chat 區塊

+ PostgreSQL SQL / views / materialized views 報表統計

+ HTML 前端

+ Excel / PPT / Report Exporter

Obsidian 暫時不納入正式架構；前端先標記為 HTML，並預留類似 Microsoft Copilot 側欄概念的 AI chat 區塊，細節後續再討論。

2. 系統總架構

啟用初期建議使用 Docker Compose 部署在一台伺服器電腦上，使用者透過瀏覽器操作 HTML 前端。



```
|
| |
| | └─ Excel Exporter
| | └─ PPT / Report Exporter
|
└─ postgres container
    └─ PostgreSQL
```

輕量版初期也可以先簡化成三個容器：

```
frontend container
backend container
postgres container
```

其中 `backend container` 先同時包含 Python runtime、匯入/清理/比對/匯出 runner、分類 worker、報表 worker 與 AI Service；正式使用以 Docker volume 掛載專案、資料與輸出目錄後執行，不以常駐 Web API 作為第一版正式入口。

資料庫 migration 不併入 app-runner 或 worker 啟動流程。即使輕量版把 runner 與 worker 包在同一個 backend image，也要保留獨立 migrate container：

```
migrate container
└─ 使用 backend image
    └─ alembic upgrade head
        └─ 跑完停止
```

此設計的目的：

```
schema 變更由明確的 migration 任務處理。
app-runner 啟動或執行任務時不自動修改資料庫 schema。
worker 啟動時不自動修改資料庫。
避免多個服務同時啟動時重複跑 migration。
server 部署時可先跑 migrate，再啟動 app-runner / worker。
```

3. 核心資料流程

整體資料流程採用「先正式入庫並保留追溯，再整理權利要求資料並建立公司名稱對照映射，再做主權項/獨立項侵權比對，最後輸出報告」的架構；報表分析保留為衍生能力，不再作為最優先主軸。

```
Backend image build
↓
migrate container
↓
alembic upgrade head
↓
PostgreSQL schema ready
↓
WIPS Excel / GPSS API / 其他專利資料
↓
Import Service
↓
Raw Layer / Core Layer
↓
Clean / Normalize / Dedup Service
↓
Company Alias / Owner Display Mapping Service
↓
Claim Comparison / Infringement Analysis Service
↓
主權項比對結果 / 獨立項比對結果
↓
AI Service / Claude 協助比對說明與報告文字
↓
HTML 前端顯示 / AI chat 區塊
↓
Excel / PPT / Report 輸出
↓
```

重要原則：

不要在匯入前做不可追溯的比對或分類
不要讓 AI 直接修改正式資料
分類與報表是衍生能力，優先順序低於侵權比對與報告輸出
前端或使用者端不直接改 PostgreSQL；正式任務透過 Docker 掛載後的 app-runner 執行
資料庫 schema migration 只能由 migrate container 執行
app-runner / worker 不得自行執行 alembic upgrade head

3.1 資料庫分層架構

資料庫與查詢設計先採四層分工。現階段先完成 Layer 1 與 Layer 2；Layer 3 與 Layer 4 先在架構上預留，等侵權比對、公司對照、runner 與前端需求穩定後再實作。

Layer 1 原始層 Raw Layer
Layer 2 正規化核心層 Core Layer
Layer 3 衍生查詢層 Derived / Analytics Layer
Layer 4 應用輸出層 Runner / Report Layer

目前執行規則：

本階段只定義資料庫層架構，不新增 table。
本階段不調整既有 table 欄位。
本階段不清空、不重寫、不修改資料庫內容。
現有資料表先歸類到 Layer 1 / Layer 2。
Layer 3 / Layer 4 只保留架構位置與責任邊界。

資料庫層整體關係：

```

Layer 1 Raw Layer
  source_files
  raw_records
    ↓
Layer 2 Core Layer
  patents
  patent_people
  patent_attributes
  patent_sources
    ↓
Layer 3 Derived / Analytics Layer
  derived tables / views / materialized views
  目前預留，尚未實作
    ↓
Layer 4 Runner / Report Layer
  Docker app-runner / Report runner / Exporter / Frontend query
  目前預留，尚未實作
  
```

Layer 1：原始層 Raw Layer

```

source_files
raw_records
  
```

責任：

保存來源檔案資訊、file_hash、匯入時間與原始列資料。
raw_records.raw_data 完整保存 WIPS / Excel 原始欄位名稱與原始值。
不做報表拆碼、不改原始欄名、不作為前端查詢最佳化表。
提供追溯、重跑清理與重建後續衍生層的依據。

Layer 1 關聯：

```

source_files.id
  
```

└ raw_records.source_file_id

Layer 2：正規化核心層 Core Layer

patents
patent_people
patent_attributes
patent_sources

責任：

patents 保存專利主資料，例如三個號碼、日期、標題、摘要、權利要求、主權項、獨立項。
patent_people 保存申請人、發明人、代理人、專利權人、受讓人、轉讓人等人與公司欄位。
patent_attributes 保存其他 WIPS 欄位寬表，包含分類、引用、同族、圖檔、連結、法律/行政欄位。
patent_sources 串接 patents、raw_records、source_files，保存 dedupe_key 與追溯關係。

Layer 2 關聯：

```
patents.id
├─ patent_people.patent_id
├─ patent_attributes.patent_id
└─ patent_sources.patent_id

raw_records.id
└─ patent_sources.raw_record_id

source_files.id
└─ patent_sources.source_file_id
```

Layer 2 寫入原則：

patents 保存專利主資料，不保存 dedupe_key、source_summary、imported_at 等系統追蹤欄位。
patent_people 保存人與公司相關來源欄位，欄位本身不可被寫成 value。
patent_attributes 保存其他 WIPS 欄位寬表，欄位即使目前空值也保留。
patent_sources 保存 dedupe_key 與來源追溯關係。
raw_records.raw_data 仍是最完整來源，不被正規化表取代。

Layer 3：衍生查詢層 Derived / Analytics Layer

目前狀態：設計中，尚未實作。

預留用途：

支援侵權比對、公司名稱顯示對照、查詢、篩選、統計與後續報表交叉分析。
可由 Raw / Core Layer 重算，不取代原始資料與核心資料。

未來可能包含：

claim_comparison_results	主權項 / 獨立項侵權比對結果
company_aliases	公司名稱對照與報表/前端顯示用標準化專利權人名稱
import_export_audit_records	匯入檔案與最終報告輸出紀錄
patent_classifications	IPC / CPC / FI / F-term / USPC 拆碼後查詢表
patent_citations	引用 / 被引用 / 自引 / 他引查詢表
patent_family_members	WIPS / EPO 同族與國家布局查詢表
report_*\ views	報表查詢 view
report_*\ materialized views	大量資料時的報表快取

Layer 3 寫入原則：

Layer 3 只能由 Raw / Core Layer 衍生或重算。
Layer 3 不作為原始資料來源。
Layer 3 可以為侵權比對整理主權項與獨立項、建立公司名稱對照映射、處理專利家族去重/不去重口徑，也可以用 PostgreSQL SQL / views / materialized views 為報表拆 IPC / CPC、整理引用、彙總申請人、計算競爭力指標。公司名稱標準化只用於報表統計與使用者端顯示，不回寫或覆蓋 Core Layer 的原始公司/專利權人欄位值。
Layer 3 的表、view、materialized view 需等侵權比對與使用者端流程確認後再建立。

Layer 4：應用輸出層 Runner / Report Layer

目前狀態：設計中，尚未實作。

預留用途：

```
Docker app-runner 查詢入口
Claim Comparison runner
Report runner
Dashboard / HTML 前端查詢
AI chat 區塊
Excel Exporter
PPT / Report Exporter
AI Service 報告文字生成入口
```

讀取原則：

```
Runner / Report Layer 優先讀 Core Layer 與 Derived / Analytics Layer。
Raw Layer 主要用於追溯與重建，不作為一般前端查詢入口。
Report Layer 不直接修改 Raw / Core 資料。
```

Layer 4 邊界：

```
資料庫容器只提供 PostgreSQL 與 SQL 查詢能力。
正式查詢、比對與匯出功能由 Docker app-runner / report runner / frontend 實作。
PPT / Excel / 最終報告輸出讀取 Core / Derived 查詢結果，不直接改 Raw Layer，輸出動作必須留下 audit record。
Claude AI Service 只能透過後端服務讀取整理後資料、比對結果或報表結果，不直接操作資料庫 schema。
```

現階段結論：

```
目前資料庫表已足夠支撐 Layer 1 與 Layer 2。
Layer 3 / Layer 4 先保留架構位置，不新增資料表或服務。
等第一版侵權比對、公司對照、匯入/匯出紀錄與 AI chat 流程確認後，再補 derived tables、views、materialized views 與 runner 入口。
```

3.2 資料庫 Migration 架構

server 化後，資料庫 schema 版本管理採用 Alembic，但 migration 執行責任必須獨立於 app-runner 與 worker。

目標架構：

```
backend image
├─ Python runtime / CLI runner
├─ worker runtime
├─ importer / report runtime
└─ Alembic migration runtime

migrate container
└─ image: backend image
   command: alembic upgrade head
   lifecycle: run once, then stop

app-runner container
└─ image: backend image
   command: run selected Python entrypoint
   不執行 migration

classifier-worker / report-worker container
└─ image: backend image
   command: start worker
   不執行 migration
```

部署順序：

```
1. 啟動 postgres container。
```

2. 執行 `migrate container:alembic upgrade head`。
3. `migrate` 成功結束。
4. 啟動 `app-runner / worker / frontend`。

設計規則：

Alembic migration 檔案放在 `backend` 專案內。
`migrate container` 使用與 `app-runner / worker` 相同的 `backend image`。
`app-runner / worker` 啟動流程不得包含 `alembic upgrade head`。
Docker PostgreSQL `init SQL` 只負責最小初始化或空資料庫準備，不作為長期 `schema` 更新機制。
既有資料庫要納入 Alembic 管理時，需先評估 `stamp head` 或專門 `migration`，不能直接覆蓋資料。

目前狀態：

此架構已列入設計，但尚未實作。
目前仍以 `sql/005_six_table_schema.sql` 與 `scripts/db_reset_and_import.ps1` 管理開發資料庫重建。
等使用者確認 `server` 內容後，再建立 Alembic 目錄、migration revision、backend image 與 migrate service。

4. Claude AI 的定位

Claude 不直接接前端，也不直接碰資料庫，而是包在後端的 AI Service 裡。

Docker app-runner / Worker

↓
AI Service

↓
Claude API

↓
後端驗證輸出

↓
PostgreSQL

Claude 主要支援後端已整理好的專利資料、PostgreSQL 統計結果與比對任務，核心用途改為協助主權項與獨立項的侵權比對說明、差異摘要、風險描述、報告文字生成，以及前端 AI chat 區塊中的問答輔助。規則分類、報表文字與 PPT 文字仍可使用 Claude，但優先順序低於 `claim comparison workflow`。

Claude 不負責資料匯入、資料清理、去重、SQL 寫入、正式判定、報表統計、PPT / Excel / 最終報告檔案產生。

AI 輸出必須採用固定 JSON schema。分類任務可使用下列格式，例如：

```
{
  "patent_id": "xxx",
  "category_key": "brushless_motor_control",
  "confidence": 0.86,
  "reason": "The patent describes control of a brushless motor drive system.",
  "needs_review": false
}
```

後端必須驗證 `category_key` 是否存在、信心分數是否合理、欄位是否完整，以及是否需要人工確認。侵權比對任務也必須以結構化結果回傳，並區分主權項比對與獨立項比對；AI chat 顯示的回答只能引用後端允許的資料範圍與比對結果，不得直接修改正式資料。分類樹調整不能由 AI 自動修改正式分類表，只能先產生 `classification_suggestions`，再經人工確認與版本化處理。

5. 報表與輸出定位

報表重要性調整為低於侵權比對與最終報告生成。第一版仍保留專利分析常用圖表與表格能力，但定位是支援使用者理解資料與補充報告內容，不再是最優先交付主軸。

保留的第一版報表內容：

1. 專利申請趨勢圖
2. 專利布局國家分析圖
3. IPC / CPC 技術分類統計圖
4. 主要申請人分析圖
5. 企業研發能量及競爭力圖
6. 高被引用專利表

資料需求：

專利申請趨勢圖

主要使用 `patents.application_date / application_year`。

專利布局國家分析圖

主要使用 `patents.country_code`，必要時再結合同族國家欄位。

IPC / CPC 技術分類統計圖

短期可從 `patents` 的 `Curr./Orig. IPC/CPC Main` 欄位與 `patent_attributes` 的 `Curr./Orig. IPC/CPC All` 欄位查詢。

若要常態支援篩選、分布、排行與交叉分析，後續應建立 `patent_classifications` 這類 `derived relation table`。

主要申請人分析圖

主要使用 `patent_people` 的申請人、專利權人等來源欄位，並透過公司對照映射產生報表/前端顯示名稱；`Core Layer` 原始欄位值不因標準化而被改寫。

企業研發能量及競爭力圖

會綜合申請量、年度趨勢、技術分類分布、同族/國家布局、引用或被引用資料。

此報表屬於複合指標，指標公式需另外定義並版本化。

高被引用專利表

主要使用引用/被引用相關欄位。

若要穩定排序與篩選，後續可建立 `patent_citations` 或 `citation metrics derived table`。

設計原則：

報表查詢不得破壞 `raw_records` 與 `patent_attributes` 的原始保存。

常態報表需要的拆碼、統計、排行，優先在 PostgreSQL 建立 `derived / relation tables`、`views` 或 `materialized views`。

`derived table / view / materialized view` 是查詢加速與報表用，不取代原始 `WIPS` 欄位；`DuckDB` 先不納入第一版報表統計路徑。

第一版先確保資料欄位來源、侵權比對依據、匯入/匯出紀錄與統計邏輯可追溯，再做視覺化與 `PPT` / 最終報告版型。

6. 第一版 MVP 開發順序

建議開發順序如下：

1. 建立 Docker Compose
 - frontend
 - backend
 - postgres
 - migrate
2. 建立 Docker 掛載式 Python 執行入口
 - container health check
 - DB connection
 - config / env
 - migration 執行邊界
3. 建立 PostgreSQL schema / Alembic migration
 - raw_layer
 - core_layer
 - derived_layer
 - app_layer
 - import / export audit records
4. 做資料庫檔案匯入
 - 上傳 WIPS Excel / 其他來源檔
 - 記錄 source file / import run
 - 保存 raw_records
 - 寫入 core tables

5. 做清理 / 去重 / 正式入庫
 - dedupe_key 去重
 - 專利家族去重口徑預留
 - patents / people / attributes / sources 追溯
6. 做公司對照表第一版
 - 專利權人名稱顯示對照
 - 公司 alias 對照
 - 使用者端與報表顯示標準化後專利權人名稱
 - 不改寫 patent_people / raw_records 的原始公司名稱值
7. 做主權項 / 獨立項侵權比對第一版
 - 主權項比對
 - 獨立項比對
 - 比對結果保存
 - 比對依據可追溯到 patent / raw record
8. 接 Claude AI Service 與 AI chat 區塊
 - claim comparison 說明
 - 差異摘要
 - 風險描述
 - 前端 AI chat 類 Copilot 區塊
 - JSON schema 驗證
9. 做最終報告匯出
 - 報告產生紀錄
 - Excel / PPT / Report Exporter
 - 匯出結果與來源資料追溯
10. 做報表 runner 與進階報告口徑
 - 專利申請趨勢圖
 - 專利布局國家分析圖
 - IPC / CPC 技術分類統計圖
 - 主要申請人分析圖
 - 企業研發能量及競爭力圖
 - 高被引用專利表
 - 可區分專利家族去重 / 不去重的報告

一句話總結：

本工具以 PostgreSQL 作為輕量正式資料庫，正式使用時透過 Docker 掛載專案、資料與輸出目錄，由 Python app-runner / worker 執行匯入、清理、比對、統計與匯出；資料正式入庫後，優先支援公司名稱顯示對照、主權項與獨立項侵權比對、匯入/匯出追溯與 AI chat 輔助分析；報表與 PostgreSQL SQL 統計保留為衍生輸出能力，最後由 HTML 前端操作並輸出 Excel / PPT / 最終報告。